

Embedded Systems Security: from Threat Modeling to LLM-Assisted Vulnerability Discovery

Data and Network Security

Ralli Lorenzo
PhD in Cybersecurity



SAPIENZA
UNIVERSITÀ DI ROMA

\$ whoami



Education



- B.Sc. Computer Engineering
- M.Sc. Cybersecurity
- Ph.D. Cybersecurity



Achievements



- CyberChallenge.IT 2024 Winner
- Leonardo CyberX 2025 Winner



Research Interests



- Firmware Analysis
- LLMs
- Software Testing



Ongoing Work



- **Bullseye**
Directed Firmware Fuzzer
- **CPUSPEC2SMT**
ISA-Spec -> SMT-Lib

Outline

CHAPTER



Background

Embedded Systems
Firmware
Threat Modeling
Case Studies

You are here

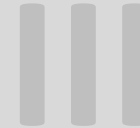
CHAPTER



Firmware Analysis

Analysis Pipeline
Firmware Rehosting
Open Challenges

CHAPTER

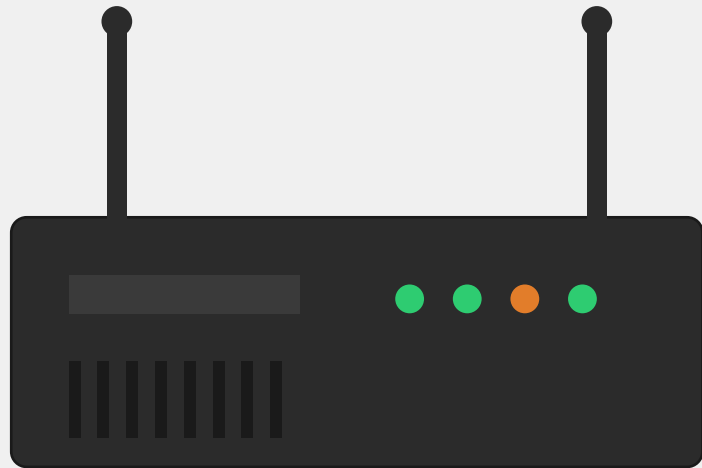


Firmware Fuzzing

Fuzzing Overview
Firm-AFL
Bullseye

More than just a Box...

EXAMPLE



A typical home Wi-Fi router

You definitely have one in your home right now.

And yet, few people ever stop to ask what's running inside.



It runs **FIRMWARE**

A small, specialized piece of software built into the device



It handles your **TRAFFIC**

Every website you visit, message you send, etc. passes through it



It's almost always **ONLINE**

It's connected 24/7, and it is rarely updated

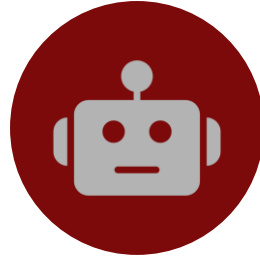
Today's question: *how secure is the code running inside the devices we rely on?*

What if the device is compromised?



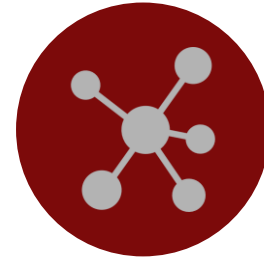
STEAL Data

Passwords, browsing history, private communications



Use it as a BOTNET

Your router joins thousands of others to attack hospitals, banks, and infrastructure



LATERAL movement

Compromise other devices on the network and spread the attack



Compromising one device can result in a foothold into an entire network

What is an Embedded System?

A small computer built into a larger device, designed to perform **specific functions** reliably over long periods. When these devices are connected to the internet, they form the **Internet of Things (IoT)**.

EXAMPLES YOU ALREADY INTERACT WITH



Wi-Fi Router

in your home



Smart TV

in your living room



Security Camera

on your front door



Car ECU

in every modern car



Industrial PLC

in factories & utilities



Pacemaker

inside a human body

← *everyday & consumer*

safety-critical →

General-Purpose VS Embedded Systems

General-Purpose Systems

Purpose

Runs many user-selected programs

Hardware

Powerful hardware (RAM, storage, rich I/O)

Software & Updates

Full OS (Linux, Windows, macOS), frequent updates

Lifetime & Reliability

Short lifecycle (3-7 years), ongoing security support

Embedded Systems

Purpose

Runs one dedicated function reliably

Hardware

Resource-constrained (KB-MB RAM, minimal storage)

Software & Updates

Bare-metal/minimal OS, rare or no updates

Lifetime & Reliability

Long lifecycle (10-25 years), security often limited

Firmware

Low-level software that is permanently stored on the device and **directly controls** its hardware.

Software layer stack

User / Applications

OS / Kernel

► **FIRMWARE** ← (often the **ONLY** layer)

Hardware (CPU, RAM, sensors)

Why firmware matters



Stored in flash memory

Persists even after power loss



Executes at boot

Often before, or instead of, an OS



Hard to replace or update

Often requires physical access or special tools

Problem Statement

Embedded devices are more widespread than ever, serving as **foundational nodes of the IoT ecosystem**.

Due to **resource and architectural constraints**, these devices rely on little to no abstraction layers, often leaving firmware as the sole software layer.

This makes firmware the **primary attack surface**, and its analysis increasingly critical:

17B+

IoT devices
deployed globally

97%

of firmware images
never patched

1 in 3

firmware images
contain known CVEs

Common Embedded Systems Vulnerabilities



Insecure Network Services

Unnecessary open ports, weak protocols, unencrypted interfaces



Hardcoded Credentials

Default embedded passwords baked into firmware binaries



Command Injection

Unsanitized user input passed directly to system calls



Buffer Overflows

Unchecked memory copies that overwrite stack or heap



Outdated Components

Reuse of vulnerable third-party libraries with known CVEs



Exposed Debug Ports

They grant full shell access or firmware extraction to physical attackers.

Threat Model

*Before defending a system, understand **who** you're defending it against.*



Threat Model - WHO

Remote attacker

No physical contact. Reaches the device across the internet.

EXAMPLES

Someone scanning **for devices with default passwords** · A ransomware group targeting a class of routers · **A nation-state** pre-positioning malware in critical infrastructure

Physical attacker

Has the device in hand. Can open it and probe its hardware.

EXAMPLES

A **border inspection** · A researcher buying a device to reverse-engineer it · A **stolen or lost device** · A malicious repair shop

Insider / supply chain

Trusted actor with legitimate access to the device before you get it.

EXAMPLES

A **disgruntled employee** · A compromised firmware vendor · A **contractor** with factory access · Backdoored chips from an untrusted supplier

Threat Model - WHAT

Steal data

Extract information the device holds, processes, or transmits.

EXAMPLES

Wi-Fi passwords · **camera & microphone feeds** · payment card data · **trade secrets** from industrial sensors · medical records from health devices

Control the device

Turn it into a tool that works for the attacker, not its owner.

EXAMPLES

Make it a **botnet node** · use it as a **pivot into the rest of the network** · spy on its owner · mine cryptocurrency · serve malware

Disrupt or destroy

Take the device (or what it controls) offline or cause it to run outside its specifications.

EXAMPLES

Brick the device · **sabotage industrial machinery** · disable safety systems · **cause physical harm** via cars, medical devices, or critical infrastructure

Threat Model - HOW

Network interface

Services the device exposes to the outside world.

EXAMPLES

Exposed **web admin panels** · unpatched protocol bugs (HTTP, Telnet, UPnP) · **default or hardcoded credentials** · open ports left on by accident

Hardware debug ports

Interfaces left enabled from the factory – a physical attacker's shortcut.

EXAMPLES

UART, JTAG, SPI for dumping firmware · chip-off attacks on flash memory · **side channels** (power, timing, EM) that leak secrets without breaking the device

Firmware updates

The update channel is itself an attack surface.

EXAMPLES

Unsigned updates accepted by the device · **compromised update servers** · **rollback attacks** to a known-vulnerable version · malicious vendor insider pushing bad firmware

Applying the Threat Model: Home Router

Target: a standard home Wi-Fi router running embedded Linux firmware



Scenario A: Remote Attacker

Attack Path

1. Router's web admin interface is exposed to the internet
2. Attacker finds a buffer overflow bug in the HTTP handler
3. Sends a crafted packet to gain remote code execution

They can now spy on your network traffic



Scenario B: Physical Attacker

Attack Path

1. Attacker opens the router's casing
2. Connects to a debug port (UART) on the circuit board
3. Dumps the full firmware image
4. Analyzes it offline to find hardcoded credentials or bugs

They can now attack every router of this model

Case Studies: when firmware fails

2010

Stuxnet

A nation-state worm that physically destroyed uranium centrifuges in Iran.

WHO

Nation-state attacker with insider reach

WHAT

Sabotage Iranian uranium enrichment

HOW

Malicious control-system firmware + 0-days vuln.

2015

Jeep Cherokee Hack

Researchers remotely took over a moving car through its infotainment system.

WHO

Security researchers (publicly)

WHAT

Demonstrate remote control of a moving vehicle

HOW

Entered via the entertainment system

2016

Mirai Botnet

Hundreds of thousands of hacked IoT devices took down large parts of the internet.

WHO

Opportunistic remote attackers

WHAT

Build a botnet large enough to rent out

HOW

Brute-force 62 common default credentials

*Across all three cases, the attack surface was the same: **the firmware running on the device.***

Key Takeaways

1

Embedded systems are everywhere

From home routers to pacemakers; Mostly invisible, rarely considered.

2

Firmware is often the only software layer

No OS, no sandboxing, no safety net → it is considered the primary attack surface.

3

Firmware is often insecure and rarely updated

17B+ devices deployed, 97% never patched, 1 in 3 with known CVEs.

4

Threat models make security tractable

Who attacks, what they want, and how.

5

Attackers actively exploit firmware

Stuxnet, Jeep, Mirai - firmware flaws with physical, financial, and global consequences.

Outline

CHAPTER



Background

Embedded Systems
Firmware
Threat Modeling
Case Studies

CHAPTER

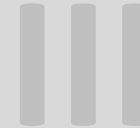


Firmware Analysis

Analysis Pipeline
Firmware Rehosting
Open Challenges

You are here

CHAPTER



Firmware Fuzzing

Fuzzing Overview
Firm-AFL
Bullseye

Problem Statement



How do we find vulnerabilities in embedded systems?

What we know:



Firmware is the main attack surface



Firmware bugs get rarely patched

What we are working with:



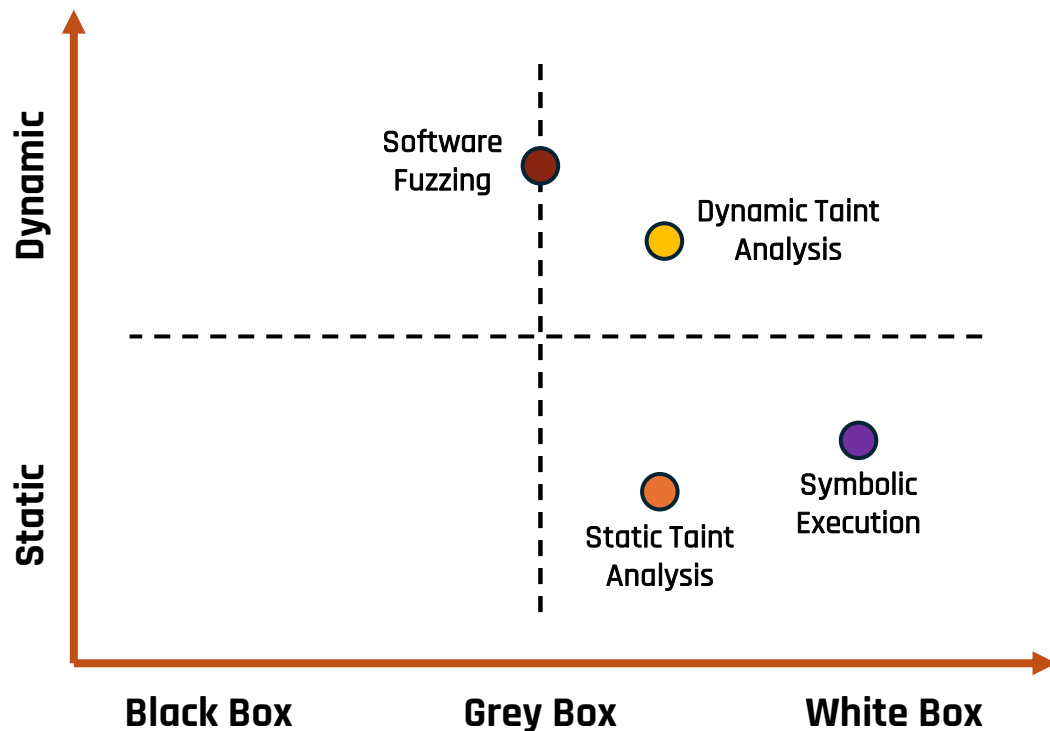
No source code available



No standard way to run the firmware

A bit of terminology...

Software testing techniques can be classified along two orthogonal dimensions:



- **Approach:**
 - **Static:** code is not executed.
 - **Dynamic:** program is executed.
- **Runtime inspection capabilities:**
 - **Black Box:** only external behavior (e.g., output, crash).
 - **Grey Box:** basic info about execution state (e.g., code coverage).
 - **White Box:** full info about execution state (e.g., line-by-line behavior).

What do we start with?

What the analyst has:

1. A firmware image

A raw binary file, usually a few MB in size.

2. No documentation

No manual, no comments, no symbol names.

3. Unknown architecture

Could be ARM, MIPS - must figure it out first.

firmware.bin

```
4F 45 4C 46 02 01 01 00 00 00
00 00 02 00 28 00 00 00 01 00
34 00 B8 16 00 00 00 00 00 00
```

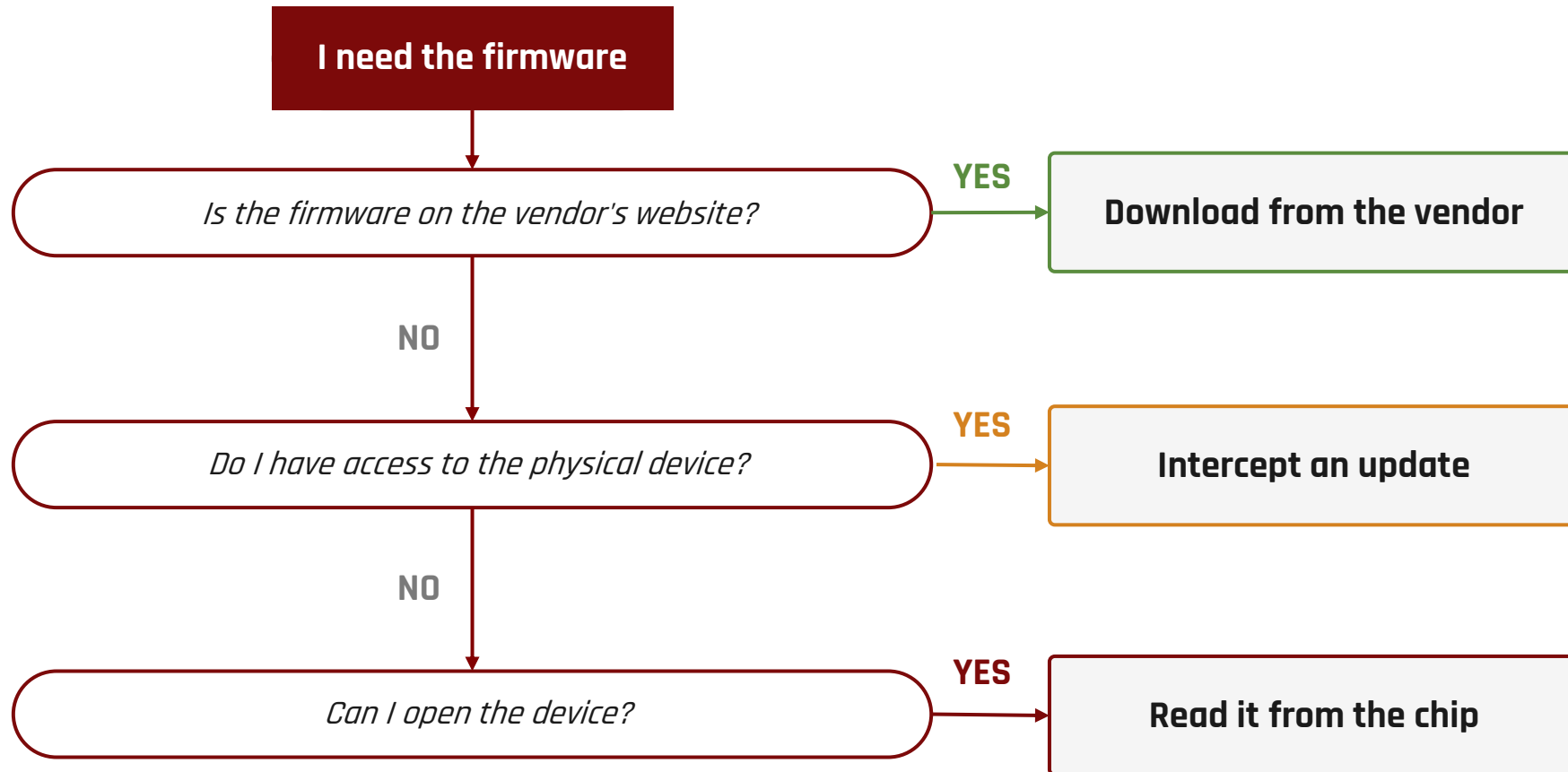


What's in here? How does it work?

*In other words: a textbook **black-box** setup*

Step 1: Extraction

How do we get the firmware off the device in the first place?



Step 2: Unpacking

How do we turn this blob into files we can read?



Inside the filesystem we find:

Binaries
httpd, cgin, busybox...

Config files
Passwords, keys, network settings

Scripts & libraries
Shell scripts, shared .so files

*How do we actually **analyze** these?*

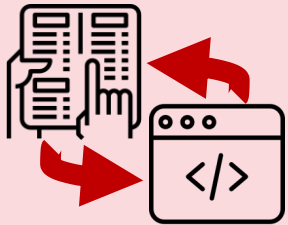
Step 3: Analysis

White Box



Static Analysis

Inspects firmware **without execution**; Analyzes decompiled/disassembled code to detect vulnerabilities. Fast and scalable, but hampered by obfuscation and missing symbols.



Hybrid Analysis

Combines static and dynamic approaches by using runtime data to validate statically-detected vulnerabilities, improving both coverage and accuracy.

Black Box



Dynamic Analysis

Runs firmware on hardware/in an emulated environment to **observe runtime behavior**. High-fidelity, but requires accurate peripheral emulation and a fully bootable firmware image.

Firmware Analysis Roadblocks

Established software testing techniques assume conditions that firmware actively violates.

ISSUE

Fault Detection

No OS -> no standard crash signals.

Scalability

Limited parallelization, scarce hardware copies, and slow reset times.

Emulation

Peripheral dependencies make full firmware emulation rarely feasible

MITIGATION

Behavioral Monitoring

Detect failures by observing responses to crafted inputs.

Full Emulation

Run firmware in a virtual environment to favour scalable fuzzing campaigns.

Partial Emulation

Redirect peripheral interactions to real hardware

Firmware Rehosting

LOOKING BACK AT THE ROADBLOCKS...

Fault Detection

Scalability

Emulation



All three roadblocks ultimately depend on one question: *can we even run the firmware?*

FIRMWARE REHOSTING

Running firmware in a virtual environment without the original hardware.

Challenges of Rehosting

Why is this so difficult? Firmware is designed to run on specific, custom hardware:

Missing Peripherals

Firmware tries to talk to hardware that does not exist in the emulator

Example: reading sensor data from a peripheral that returns garbage

Proprietary Hardware

Vendors write proprietary drivers with unique memory-mapped I/O layouts

No standard documentation available

CPU Architecture

Firmware may run on ARM, MIPS, or custom cores

Each requires a different emulator setup

*Achieving full-system emulation that boots firmware correctly **is still an open research problem.***

Rehosting Frameworks

How do we run firmware without the actual device?

These tools allow you to interact with firmware as if it were a real device over the network

Firmadyne

FirmAE

Rehosting Frameworks

How do we run firmware without the actual device?

These tools allow you to interact with firmware as if it were a real device over the network

Firmadyne

Extract firmware filesystems for emulation setup

Boots Linux firmware inside a virtualized system

Provides network interface for runtime interaction

Rehosting Frameworks

How do we run firmware without the actual device?

These tools allow you to interact with firmware as if it were a real device over the network

Improves Firmadyne's boot success rate

Handles missing dependencies more robustly

Handles missing services to get firmware running

The logo for FirmAE, featuring the text "FirmAE" in a white, bold, sans-serif font. The "Firm" part is in a standard weight, while the "AE" part is significantly larger and bolder. The text is centered within a large, solid dark red rectangular background.

Why This Matters

Once firmware runs in emulation, we have a testable target, which allows us to...



Interact with firmware

Send HTTP requests, trigger login pages, probe network services - exactly as an attacker would



Test arbitrary inputs

Feed unexpected data to find crashes and edge cases



Observe real behavior

Monitor memory, network traffic, file access; Find vulnerabilities without the physical device

Still not enough...

Rehosting gets us a running target. But finding vulnerabilities manually is painfully slow...



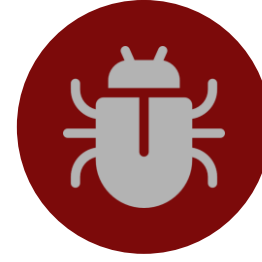
SCALING Issues

A single firmware image can take weeks to review due to the scale and complexity of its binaries



BEHAVIOURS are vast

Firmware behaves differently across vast input spaces, making it impossible to manually test all code paths



Bugs stay HIDDEN

Manual testing is never able to fully cover all edge cases

Outline

CHAPTER



Background

Embedded Systems
Firmware
Threat Modeling
Case Studies

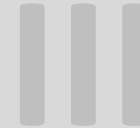
CHAPTER



Firmware Analysis

Analysis Pipeline
Firmware Rehosting
Open Challenges

CHAPTER

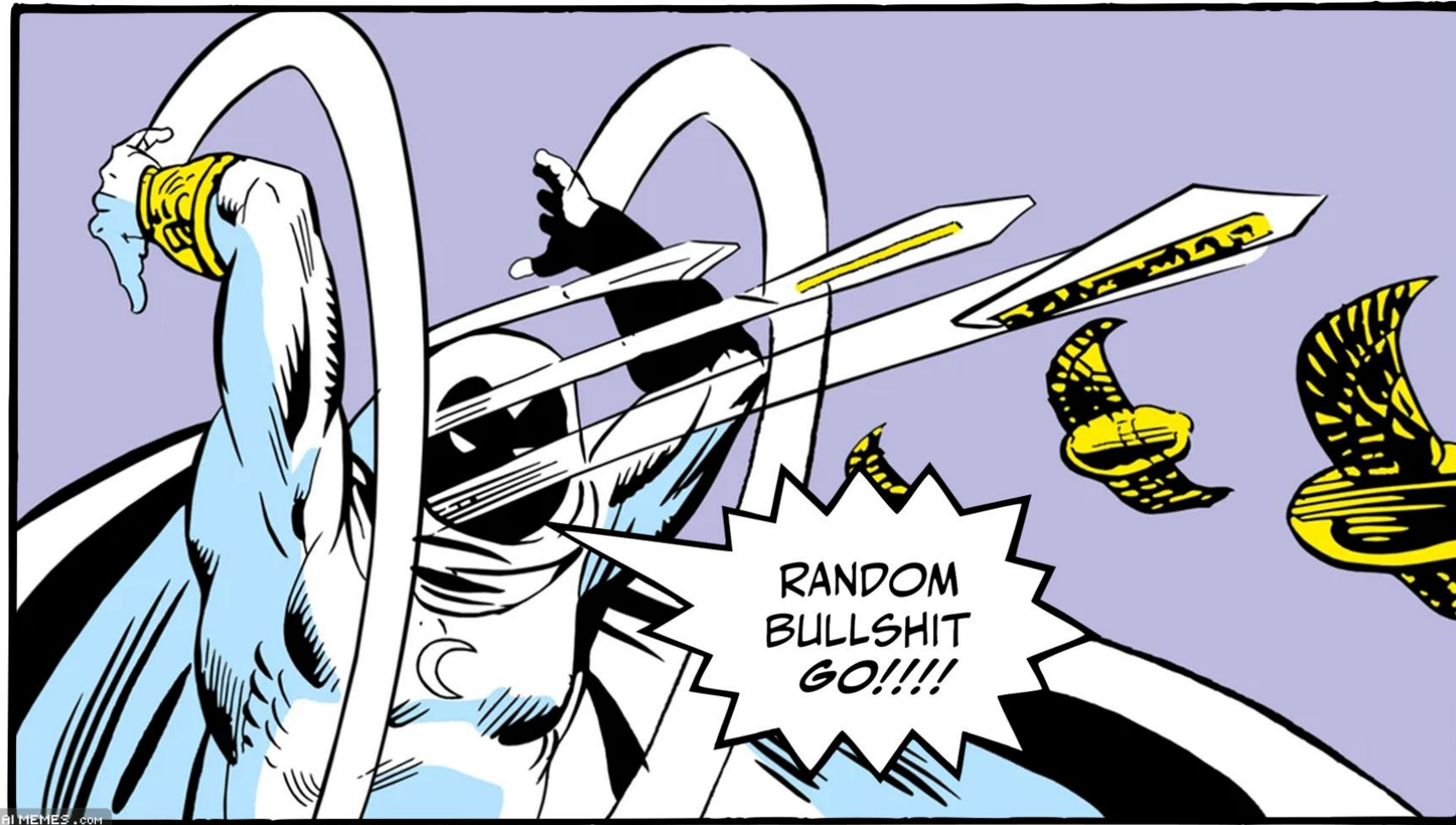


Firmware Fuzzing

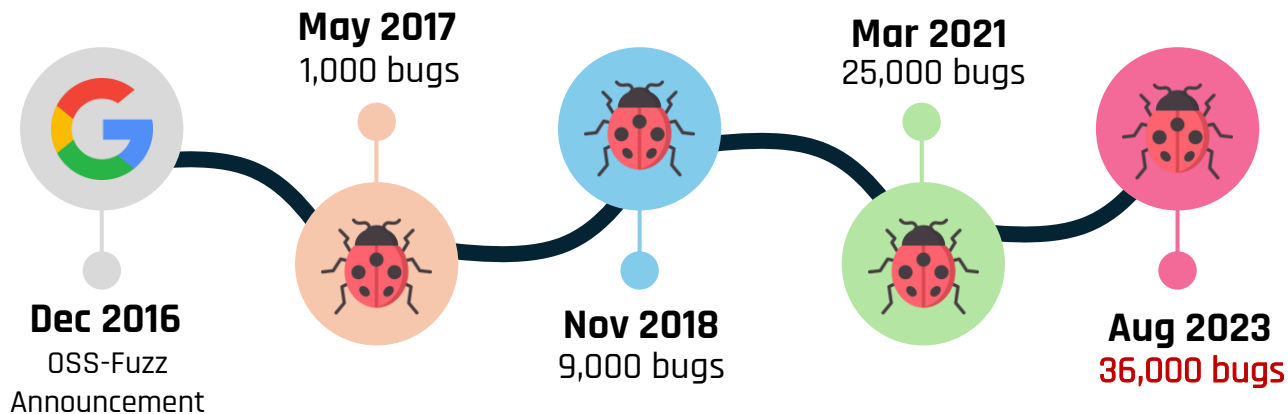
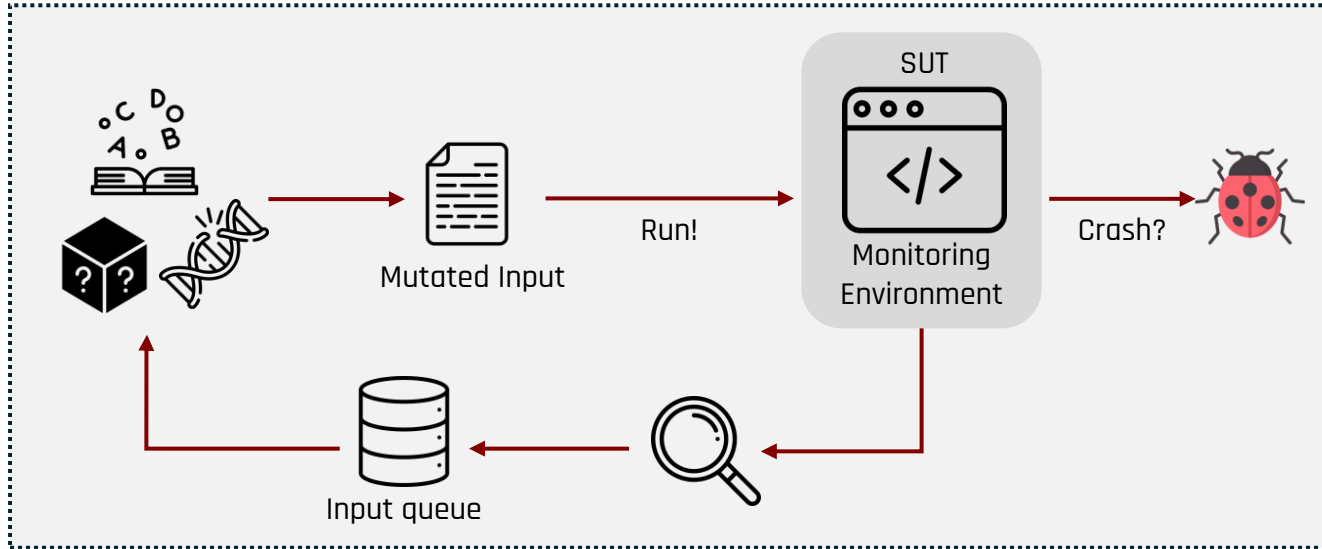
Fuzzing Overview
Firm-AFL
Bullseye

You are here

Software Fuzzing



Software Fuzzing



Core Idea

Stress target program with thousands of random inputs to trigger edge cases and uncover bugs.



Success Story: OSS-Fuzz

Google-maintained platform that fuzzes hundreds of open-source projects 24/7 to uncover bugs.

American Fuzzy Lop (AFL)

De-facto standard coverage-guided greybox fuzzer – developed by Michał Zalewski at Google in 2013. Still the baseline most modern fuzzers build upon.



Lightweight instrumentation

Compile-time hooks at every branch, ~10% slowdown.

- afl-gcc / afl-clang wrappers
- Logs edge transitions (prev_loc $\oplus$ cur_loc)
- QEMU mode for black-box binaries



Coverage Bitmap

A 64 KB shared-memory array tracking which edges fire.

- Each edge hashes to one byte
- A new bit = a new path → saved
- Drives the mutate-run-compare loop

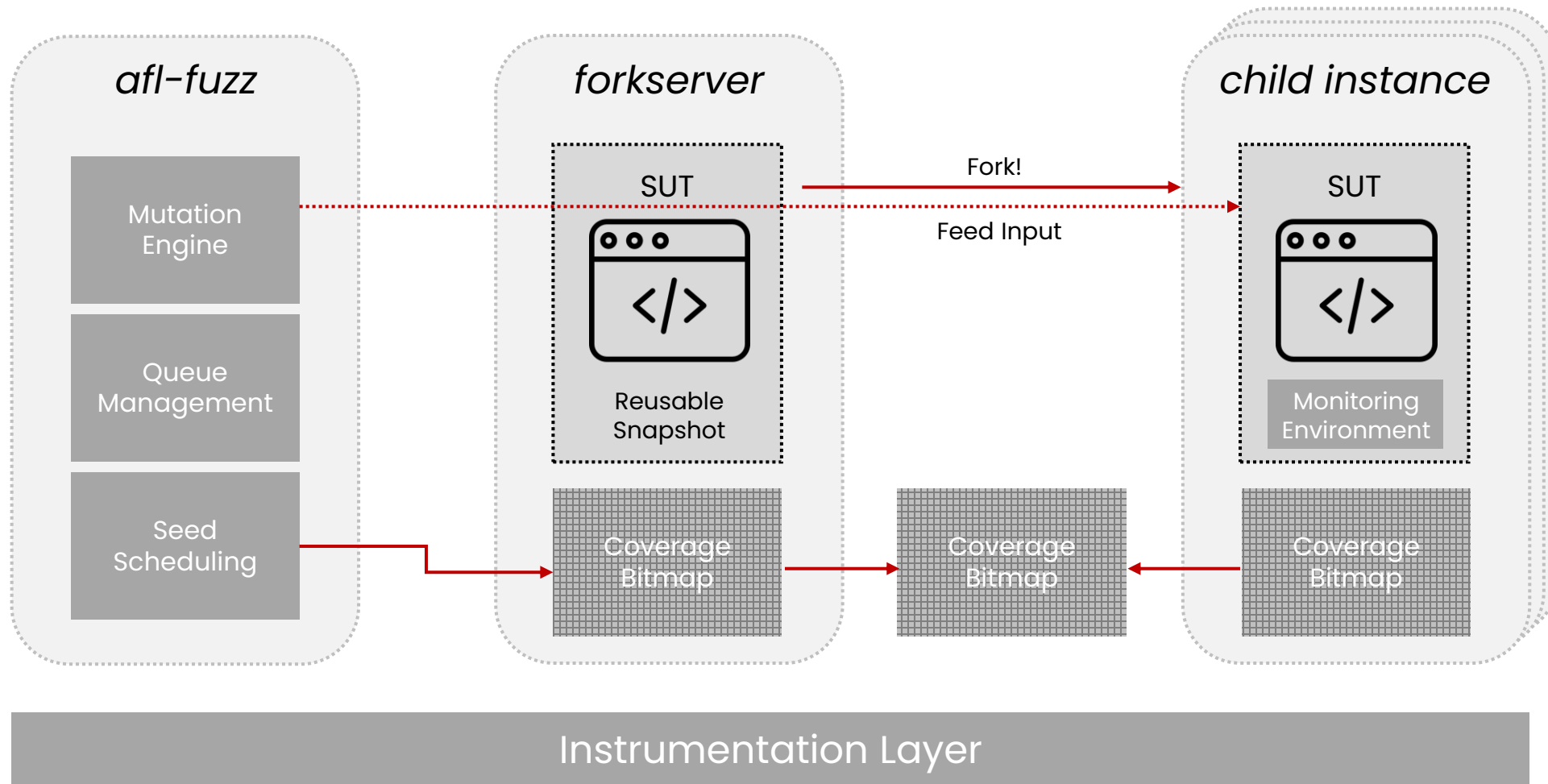


Forkserver

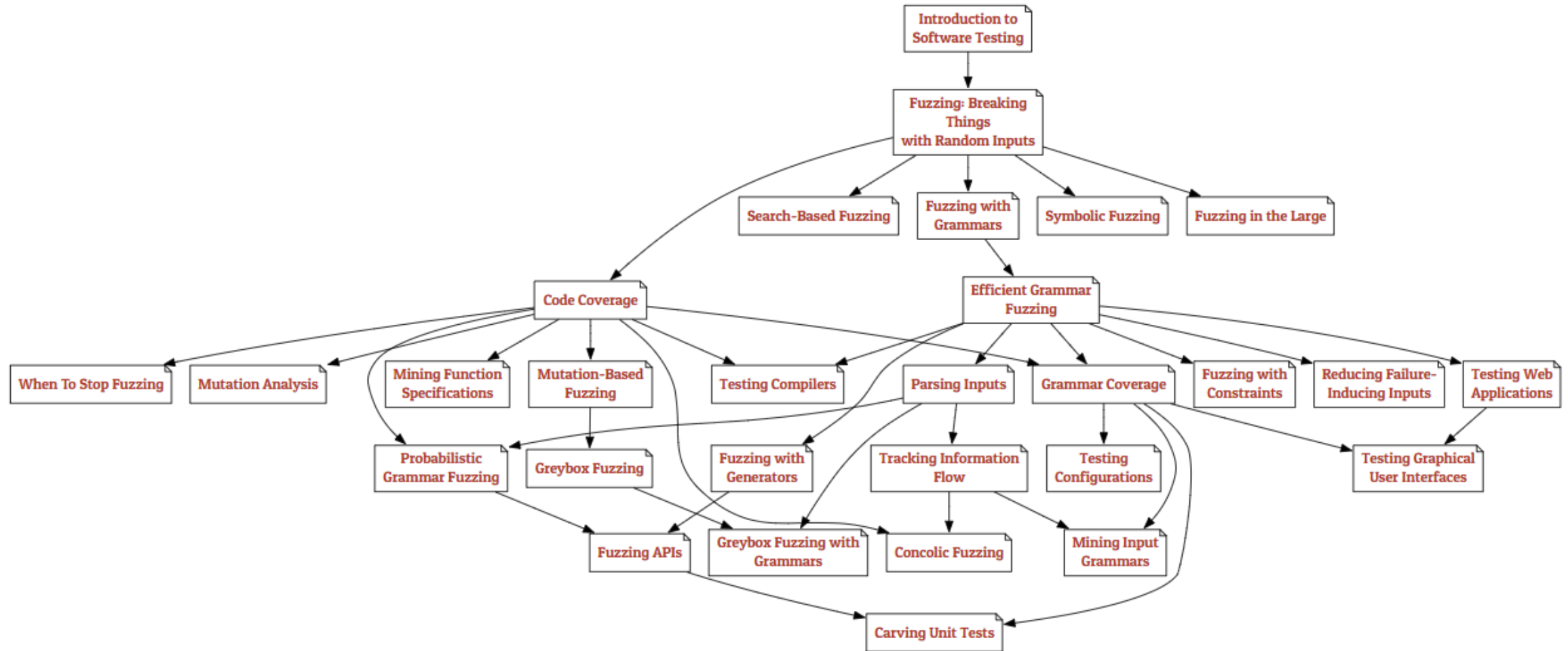
Skips process startup by forking a pre-initialized snapshot.

- No execve / linker / init per run
- Copy-on-write child, fresh state
- Typically 2-5× throughput gain

American Fuzzy Lop - Architecture



Beyond AFL: many flavours of Fuzzing

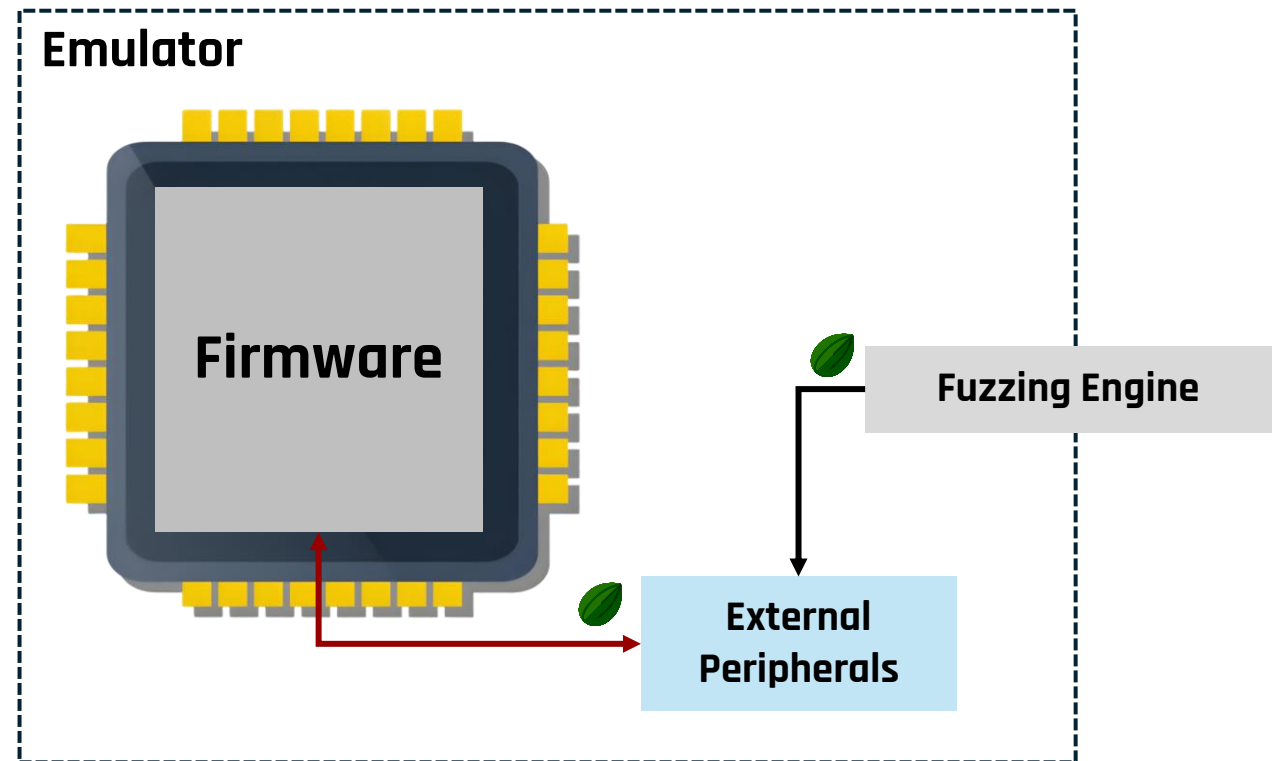


Firmware Fuzzing

Works like traditional greybox fuzzing, running firmware in an emulated environment and feeding mutated inputs to uncover vulnerabilities.

Three Ingredients

- **Emulation (Rehosting)**
Run the firmware in a virtual environment (e.g. Firmadyne, FirmAE)
- **Fuzzing engine**
Generates and mutates inputs (e.g. AFL)
- **Peripheral Handling**
Channel through which we fake the inputs to the firmware



Two Implementations

Now let's look at how firmware fuzzing works in practice.

Firm-AFL



STATE OF THE ART

Emulation-based greybox fuzzing for embedded Linux firmware.

Bullseye



MY CONTRIBUTION

Directed firmware fuzzing for embedded Linux firmware.

Two Implementations

Now let's look at how firmware fuzzing works in practice.

Firm-AFL



STATE OF THE ART

Emulation-based greybox fuzzing for embedded Linux firmware.

Bullseye



MY CONTRIBUTION

Directed firmware fuzzing for embedded Linux firmware.

Problem Statement

Approach	PROs	CONS
Executing on hardware	Compatibility	Throughput
Emulating a program	Throughput	Compatibility
Emulating full stack	Compatibility	Throughput

*Can we fuzz IoT devices with **HIGH throughput AND HIGH compatibility**?*

AFL for IoT Fuzzing?

AFL with user-mode emulation fails

Why? Unable to emulate firmware's hardware dependencies

Full-system mode works - but slower

Why? It correctly boots firmware, but introduces critical performance bottlenecks:

1

Memory Address Translation

Virtual -> physical mapping on every access

2

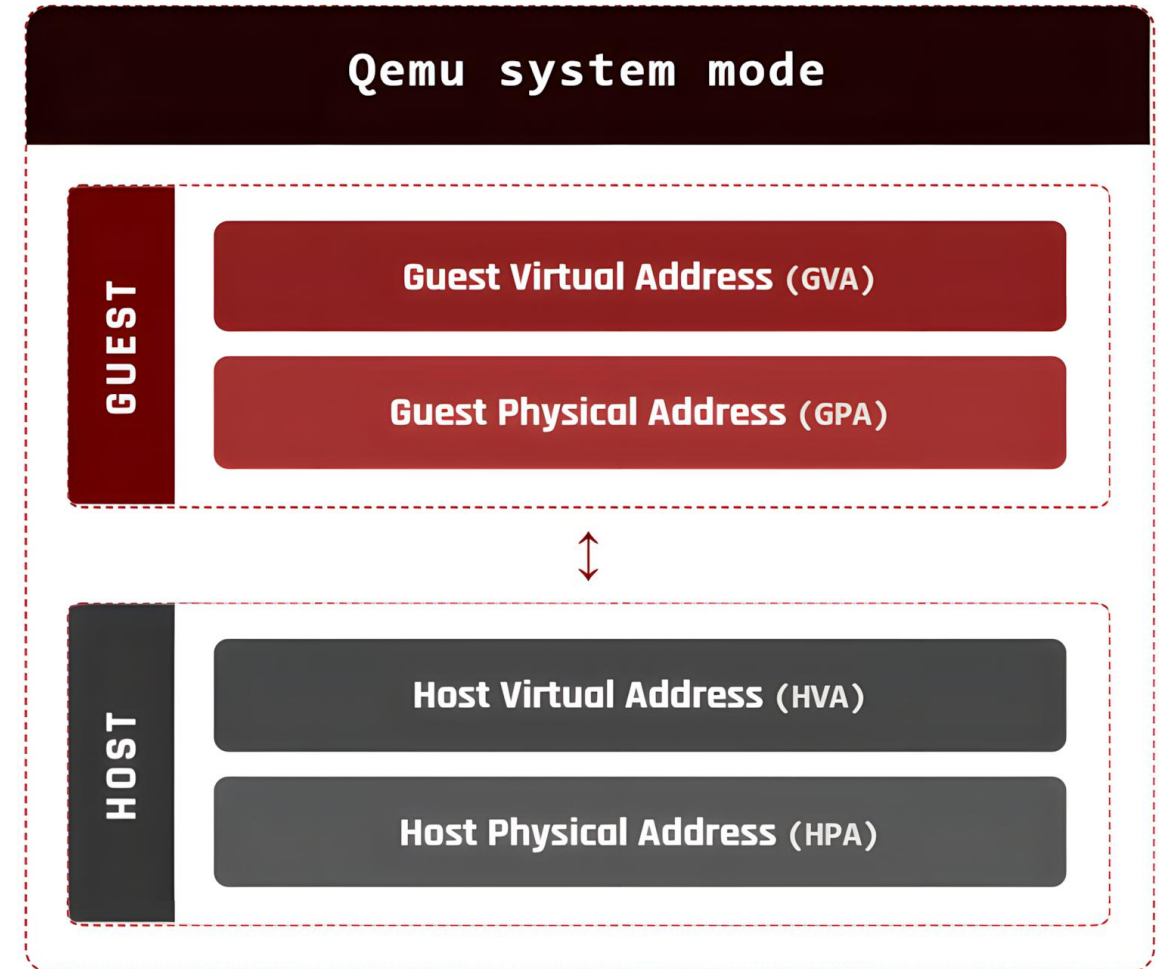
Syscall Emulation

They must be intercepted+emulated by host kernel

3

Dynamic Code Translation

Every MIPS/ARM basic block must be translated to host x86



AFL for IoT Fuzzing?

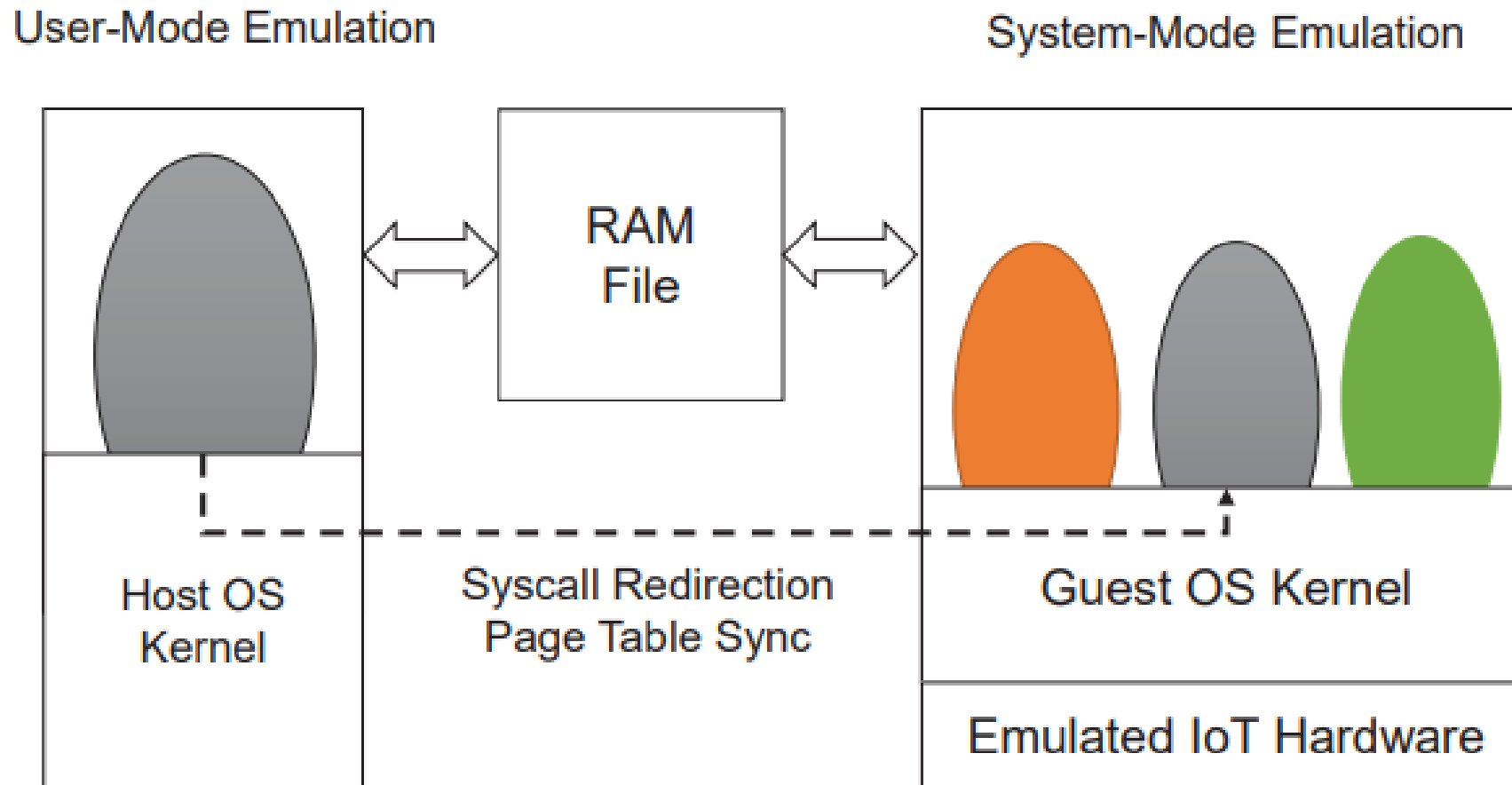
Approach	PROs	CONs
User mode emulation	Throughput	Compatibility
Full mode emulation	Compatibility	Throughput

*Why not use **BOTH**?*

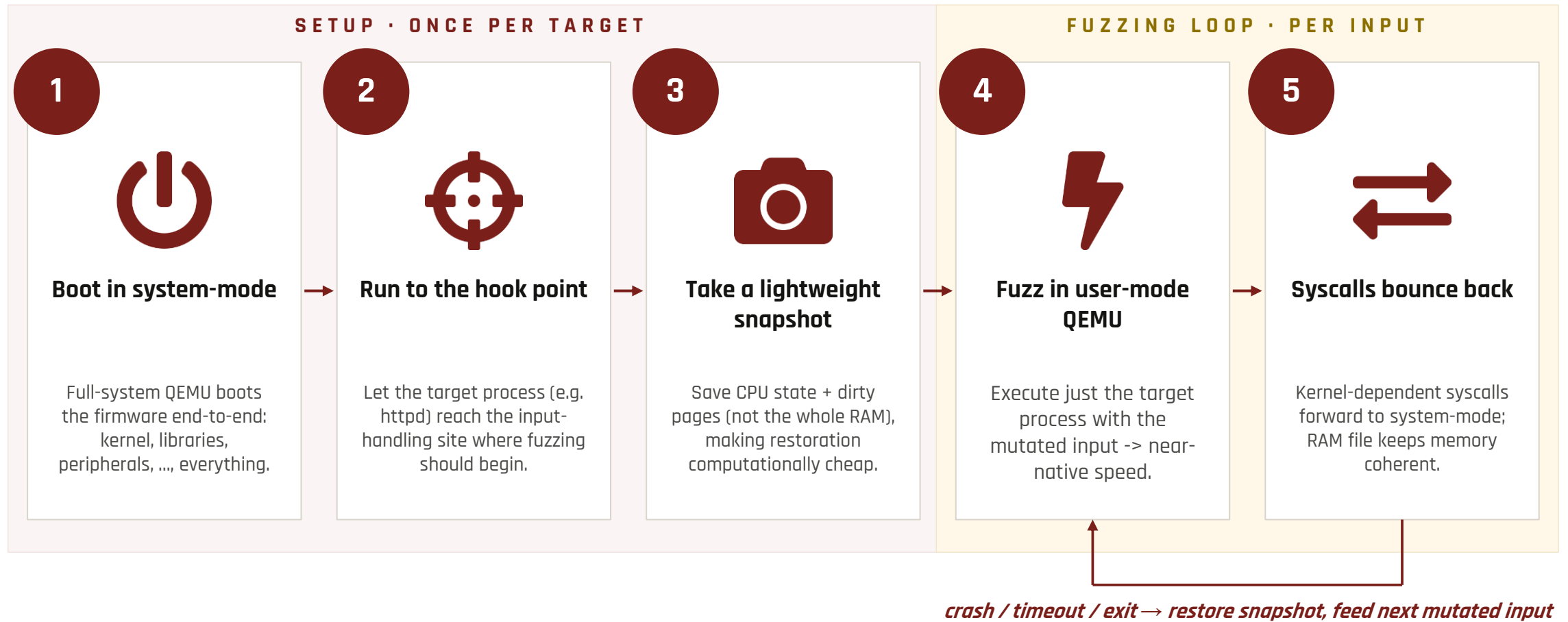
AUGMENTED PROCESS EMULATION

Fuzz the snapshotted process in user-mode; syscalls bounce back to system-mode.

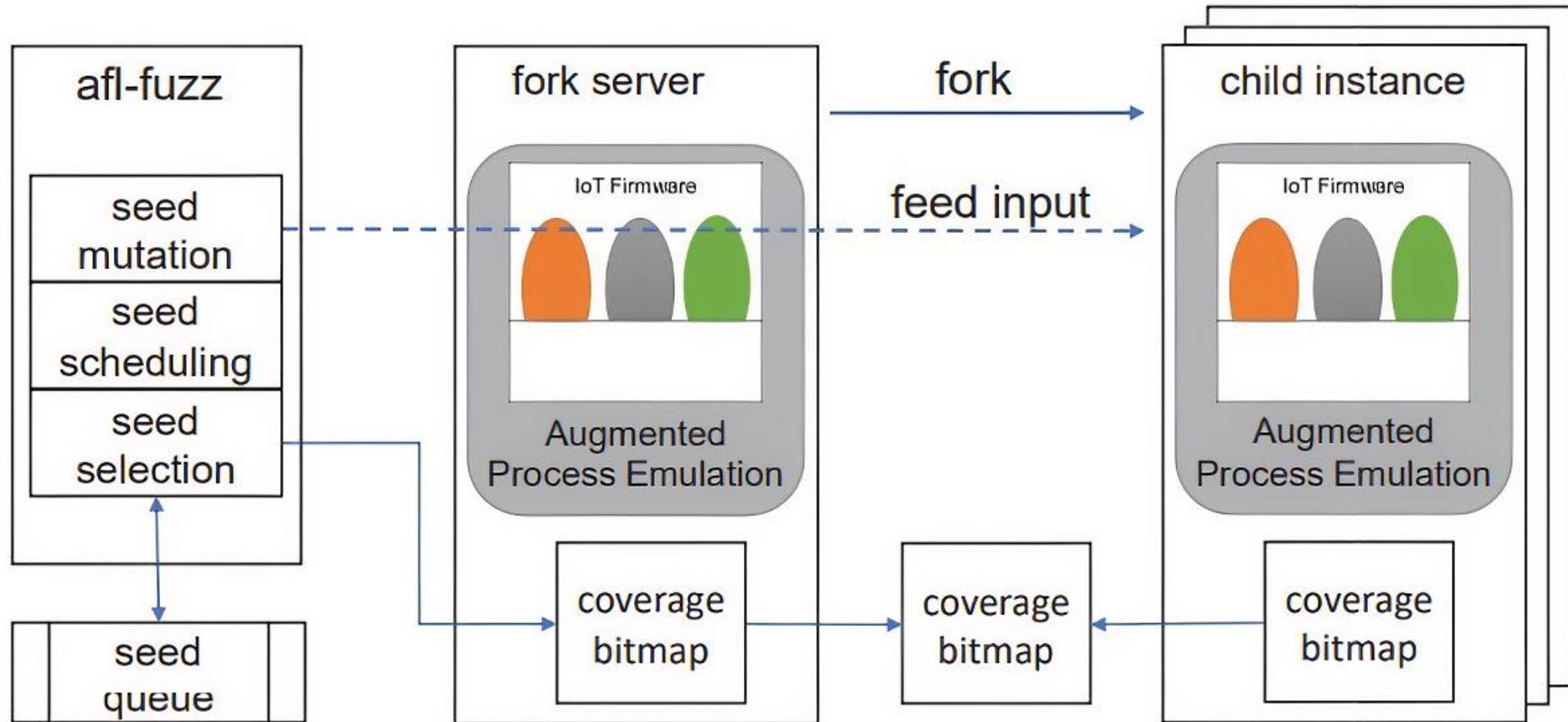
Augmented Process Emulation



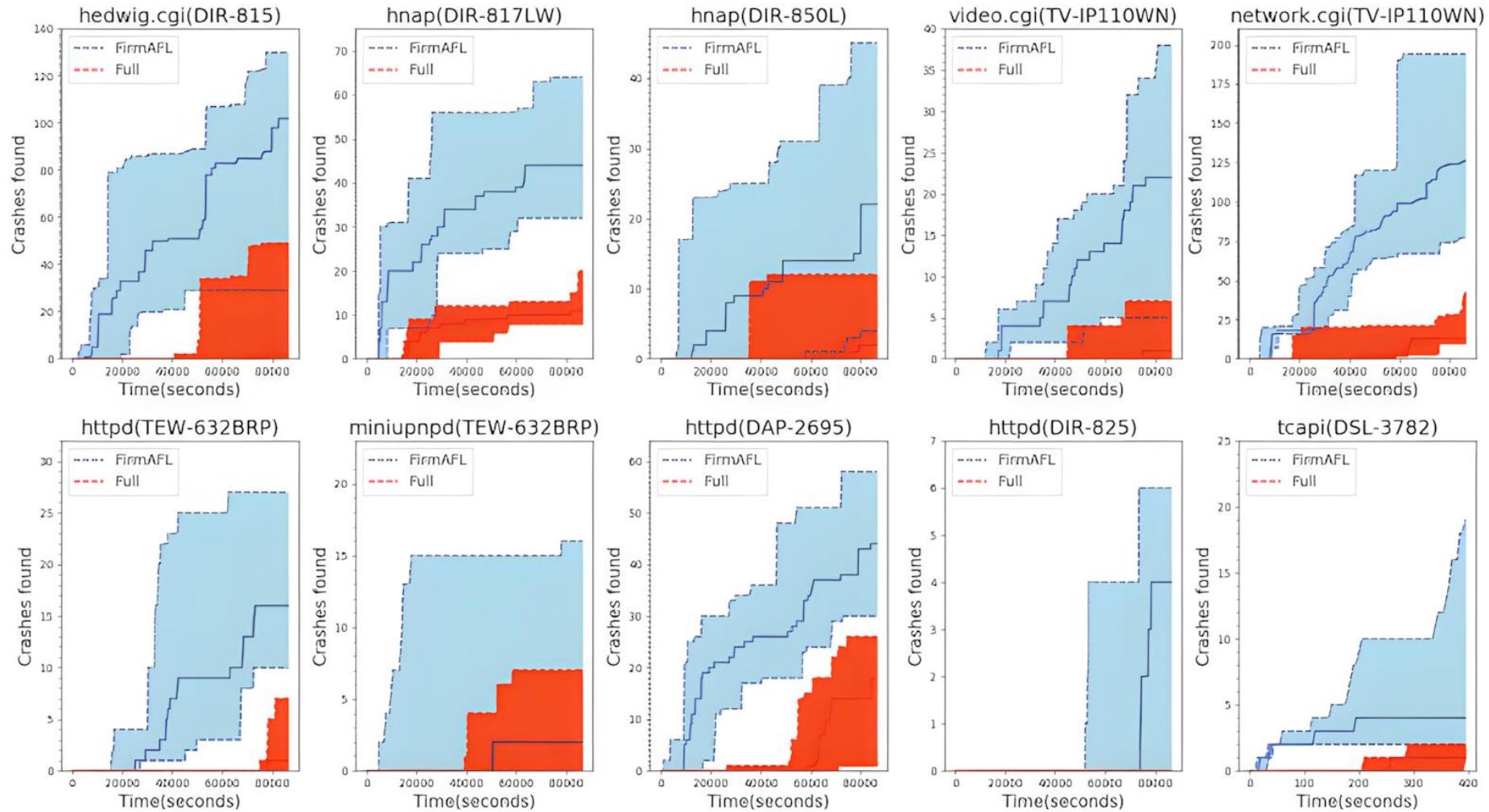
Augmented Process Emulation



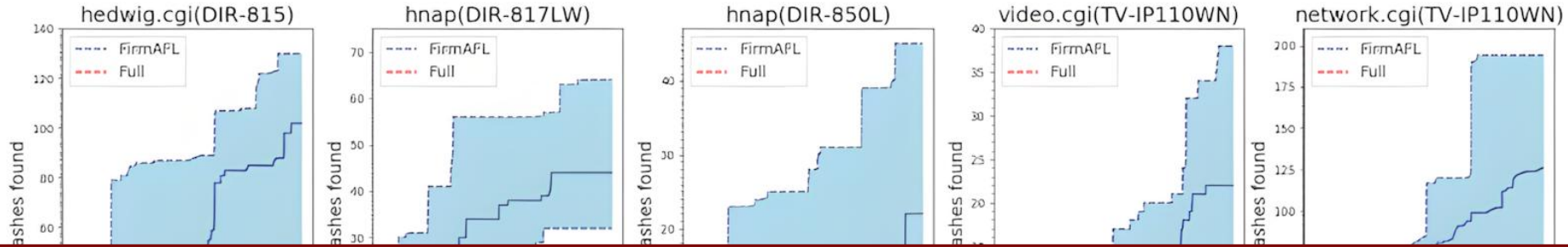
Firm-AFL Architecture



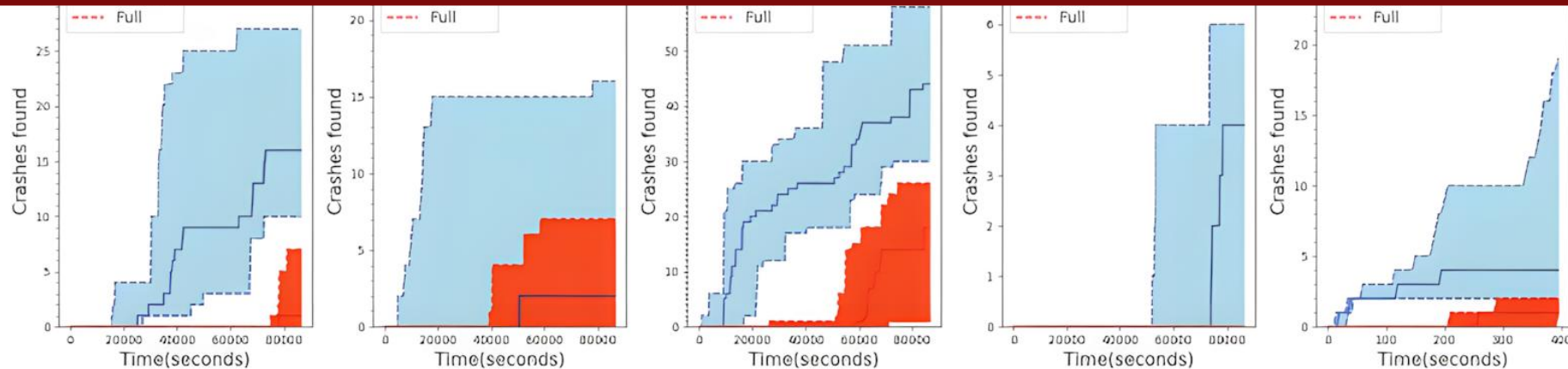
Results



Results



Firm-AFL finds significantly more unique crashes in less time. Gains are consistent across vendors, architectures, and service types.



Conclusions & Open Challenges

Conclusions



Augmented Process Emulation achieves both high compatibility and high throughput



Lightweight Copy-on-Write snapshots reduce reset overhead by 100× → more exec/s



Effective at finding vulnerabilities: 3-13x faster

Open Challenges

Only supports Linux-based firmware

Input types are limited to HTTP and environment variables

Built on vanilla AFL, making its input generation capabilities outdated

Two Implementations

Now let's look at how firmware fuzzing works in practice.

Firm-AFL



STATE OF THE ART

Emulation-based greybox fuzzing for embedded Linux firmware.

Bullseye



MY CONTRIBUTION

Directed firmware fuzzing for embedded Linux firmware.

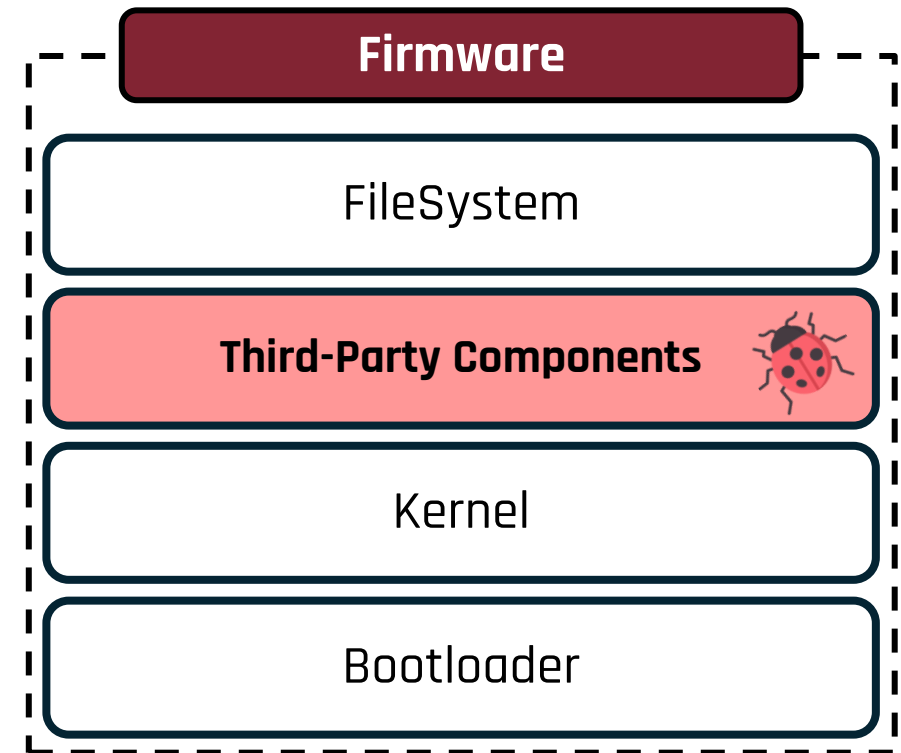
Problem Statement



Architectural constraints lead IoT device manufacturers to employ a functionality-first engineering paradigm.

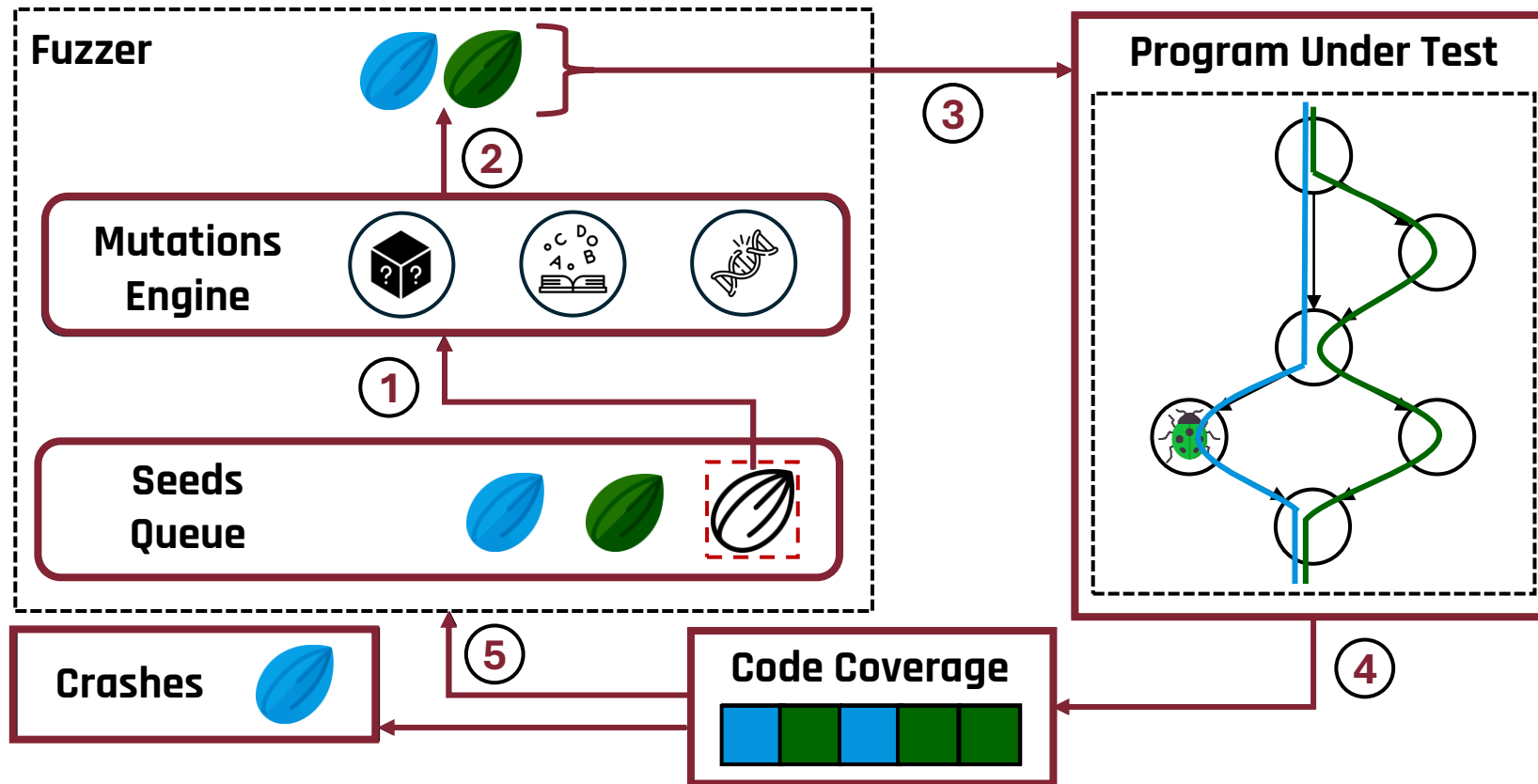
Prominent challenge: **re-use of vulnerable third-party components** in firmware development.

State-of-the-art firmware analysis practices **lack the targeted approaches** needed to effectively tackle this.



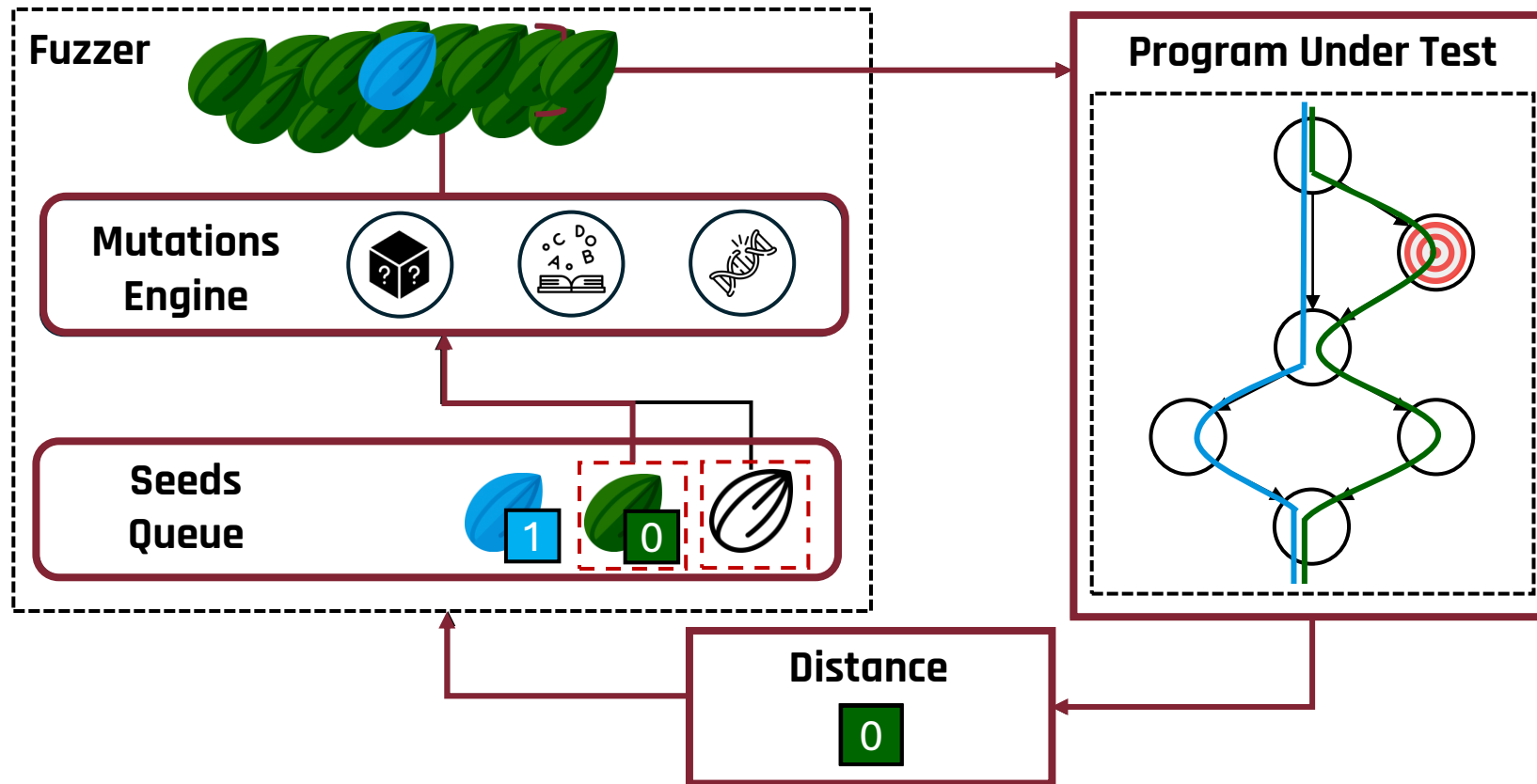
Can Fuzzing be targeted?

Coverage-guided greybox fuzzing steers effort toward inputs that explore unseen code paths.



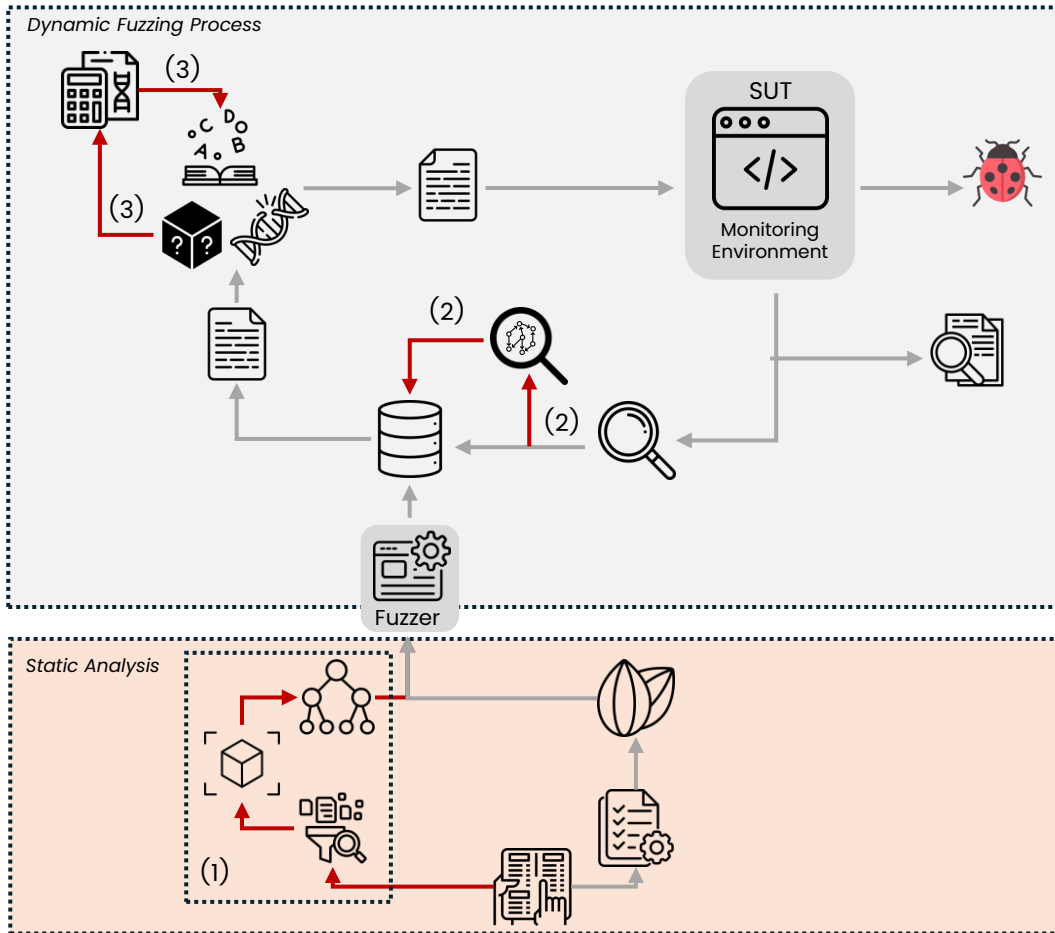
Can Fuzzing be targeted?

Directed fuzzing enables targeted exploration by prioritizing inputs that reach specific code regions.



Directed Fuzzing

Core idea: steering program execution toward a predefined set of target locations.



Reduces wasted exploration by biasing mutations toward inputs that are closer to predefined targets.

1. CG + CFGs Extraction & Targets Selection

- Extract call graph and intra-procedural CFGs
- Mark target(s) basic blocks.

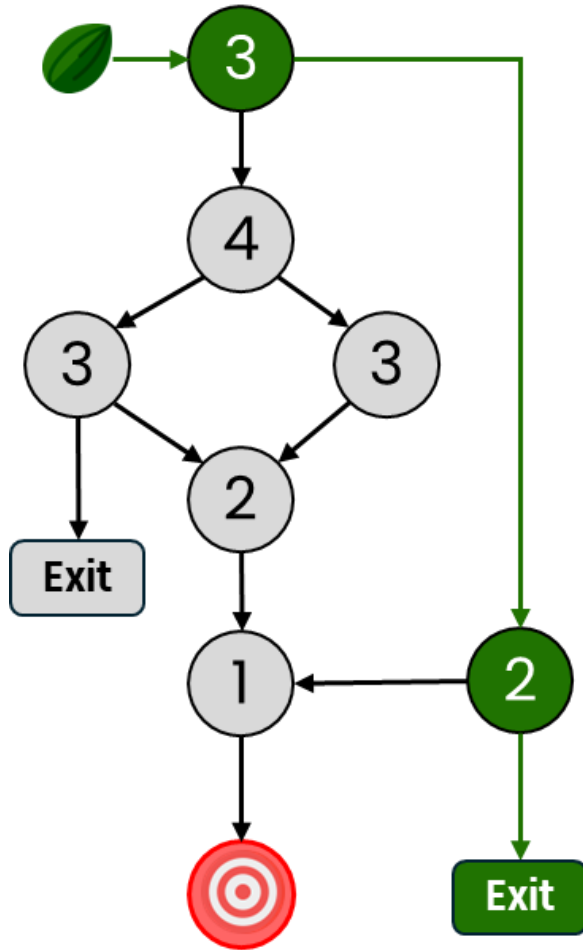
2. Distance Computation

Compute shortest-path distance (using Dijkstra) for each basic block that can reach the target(s).

3. Distance-guided Mutation Strategies

Increase time under test for inputs that execute closer to targets.

Directed Fuzzing: Distance Computation



Distance Computation

$$\sum_{i \in \text{executedBBs}} \text{score}_i = 3 + 2 = 5$$

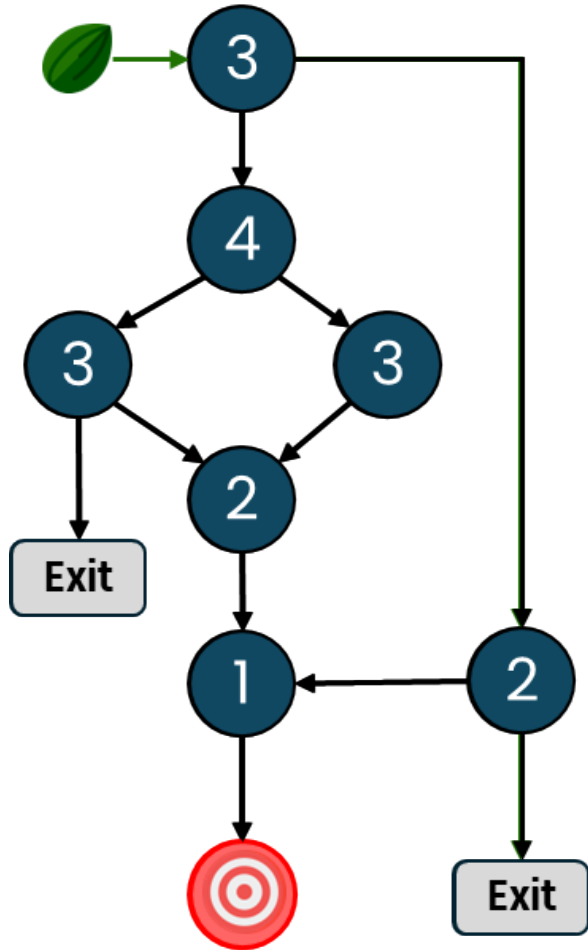
$$\text{Distance} = \frac{5}{2} = 2.5$$

Distance computation **assigns** each input a **score** by measuring **proximity to target locations** over its execution trace.

The general formula is:

$$\text{Distance} = \frac{\sum_{i \in \text{executedBBs}} \text{score}_i}{|\text{executedBBs}|}$$

Directed Fuzzing: Graphs Analysis



The proximity scores used by **distance computation** are **pre-computed statically** on the program's **control-flow graphs** and **call graph**.

Each **basic block** that lies on a path toward the target is assigned a proximity score using graph traversal analysis (Dijkstra).

Both the graph analysis and the instrumentation for runtime distance computation are performed at compile time.

State-of-the-Art

Directed Fuzzing

State-of-the-art directed fuzzer: AFLGo

Firmware Fuzzing

State-of-the-art firmware fuzzer: FirmAFL

Shortcomings

Not suitable for firmware environment.

Noisy distance metric.

Data-flow semantics not considered.

Not compatible with directed fuzzing.

Linux-based Firmware only.

Inherits limitations of traditional greybox fuzzing.

State-of-the-Art

Directed Fuzzing

State-of-the-art directed fuzzer: AFLGo

Firmware Fuzzing

State-of-the-art firmware fuzzer: FirmAFL

Shortcomings

Not suitable for firmware environment.

Noisy distance metric.

Data-flow semantics not considered.

Not compatible with directed fuzzing.

Linux-based Firmware only.

Inherits limitations of traditional greybox fuzzing.

Our Approach: Bullseye

Input Generation

Coverage-guided greybox fuzzing **struggles with enforcing input quality** → many invalid inputs are generated → **poor exploration** of the target's input space.

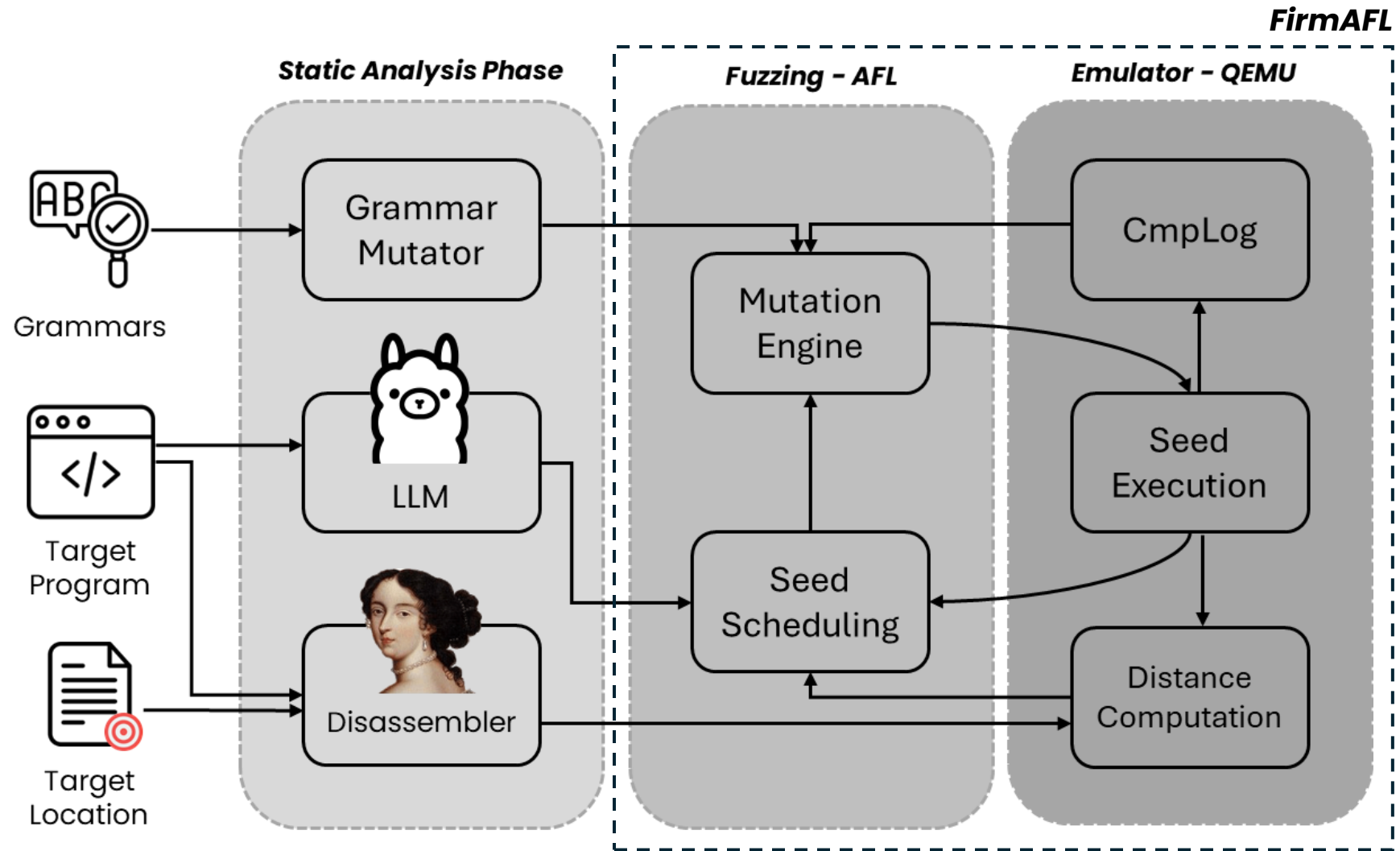
Directed Firmware Fuzzing

Existing directed fuzzers rely on **source-level instrumentation** and **static analysis on source code**, which are **unavailable** in closed-source firmware.

Distance Heuristics

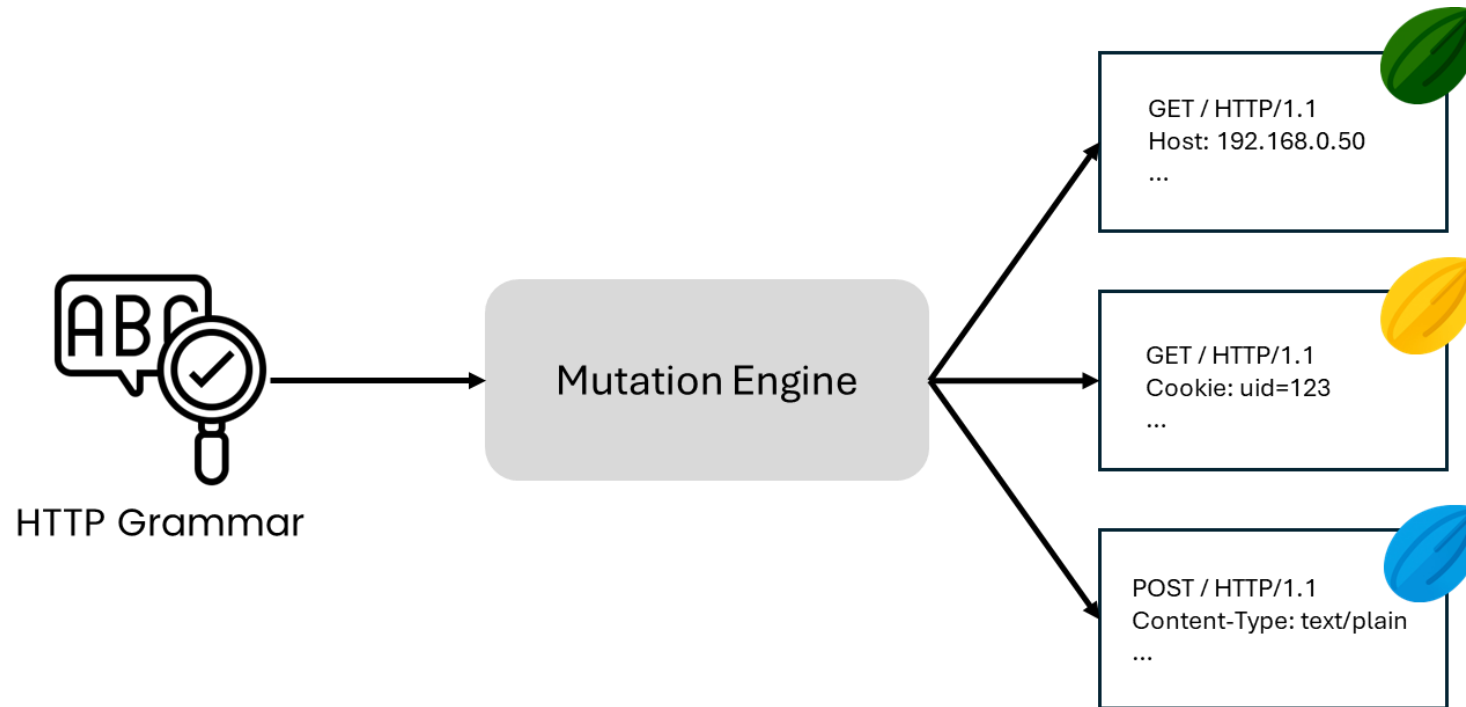
Traditional directed fuzzing heuristics are **imprecise and ignore data-flow semantics**, thus hampering the accuracy of the distance metric.

Bullseye: Implementation



Input Generation - Grammar Mutator

Component that parses user-defined grammar rules to generate structurally valid inputs.



More valid inputs → better protocol coverage → the fuzzer can reach deeper execution logic.

Input Generation - CmpLog

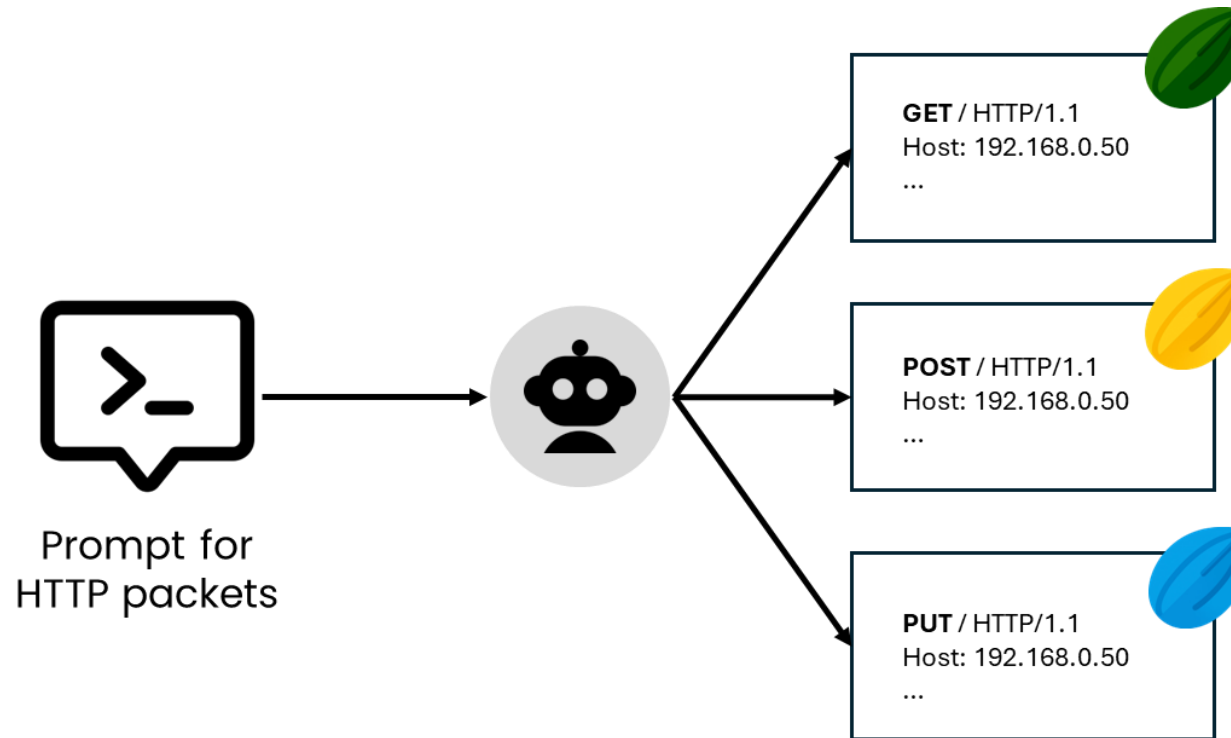
Component that logs comparison operands to guide mutations toward satisfying branch conditions.

C code	strcmp(v5, "uid", 3);	
Seed Input	TestInput	
Captured Operands	TestInput	uid
Performed Mutations	uTestInput uistInput uidtInput	uid uid uid

Hardcoded operands guide mutations → comparisons get satisfied → deeper execution paths are reached.

Input Generation - Automated Seeds Synthesis

Use LLMs to automatically generate diverse, valid seeds for the target's expected input format.



Removes manual seeds definition → **richer** initial corpus → faster, broader fuzzing campaigns.

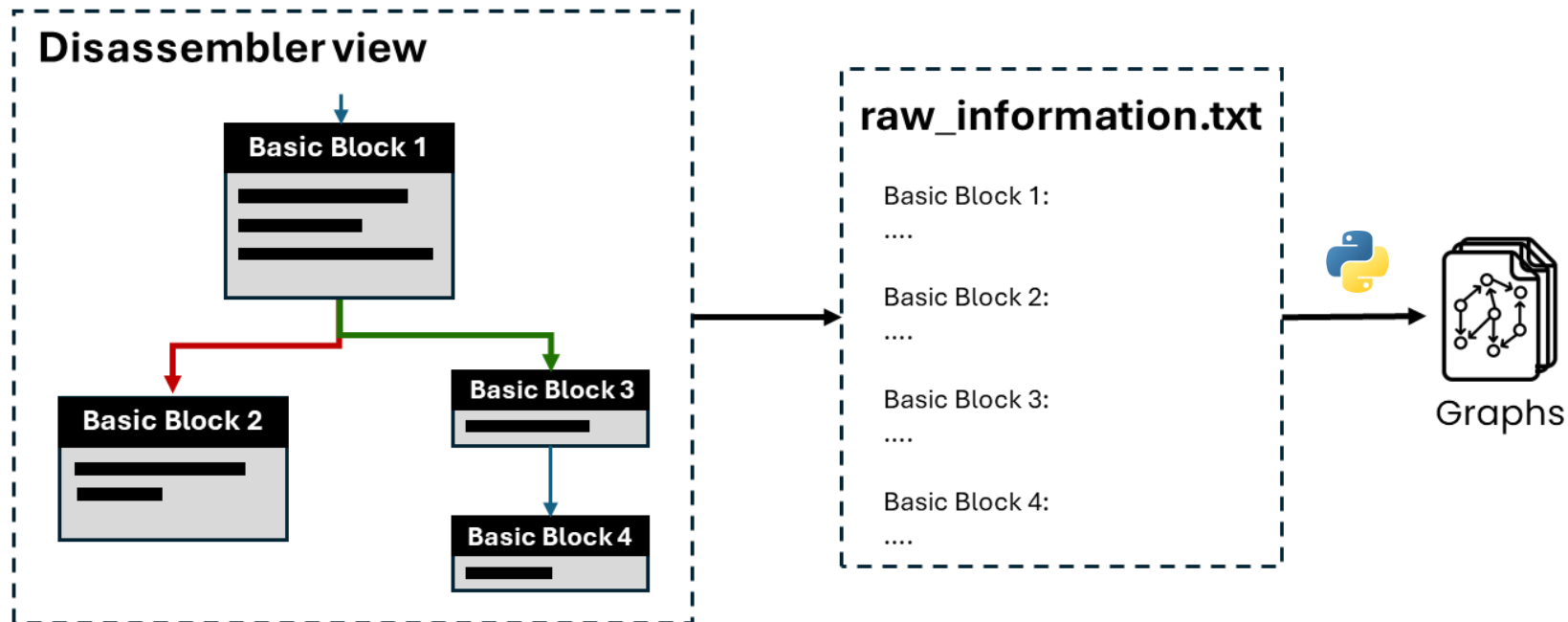
Directed Firmware Fuzzing: Graphs Extraction

Problem

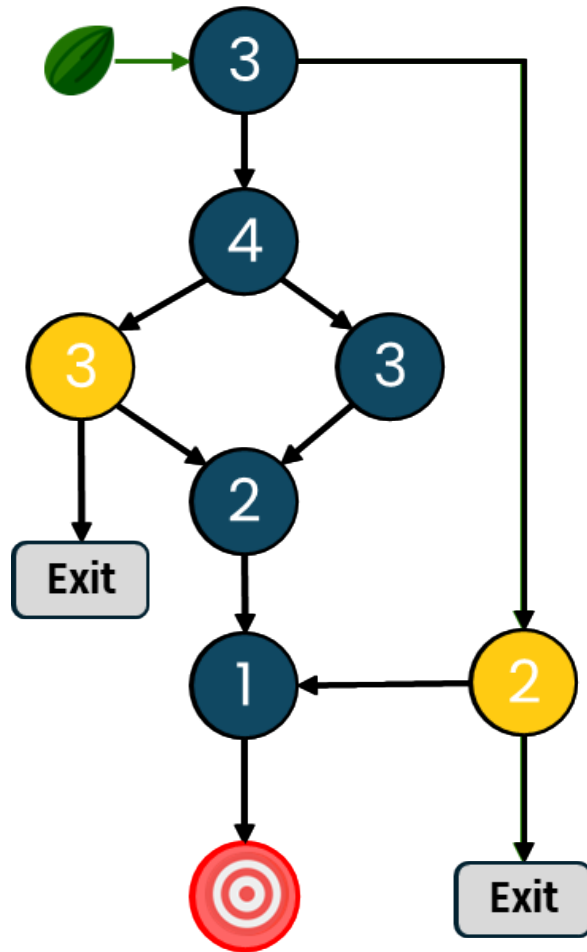
Firmware's closed-source nature prevents recompilation, making source-based directed fuzzing heuristics inapplicable."

Solution

Synthesize program graphs from basic block-level information extracted via disassemblers.



Directed Firmware Fuzzing: Noise Filtering



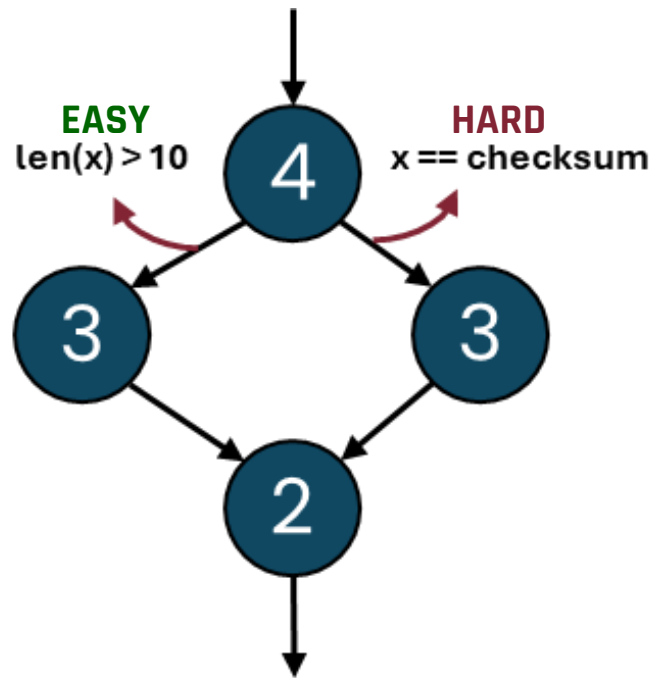
Deviation Basic Blocks

Basic blocks that lie on a path toward the target and have at least one successor from which the target is unreachable.

As these blocks are **divergence points**, they mark key decision points in target reachability.

By **considering only them** in distance computation, the **noise** introduced by other irrelevant basic blocks is **filtered out**.

Directed Firmware Fuzzing: Noise Filtering



Conformance

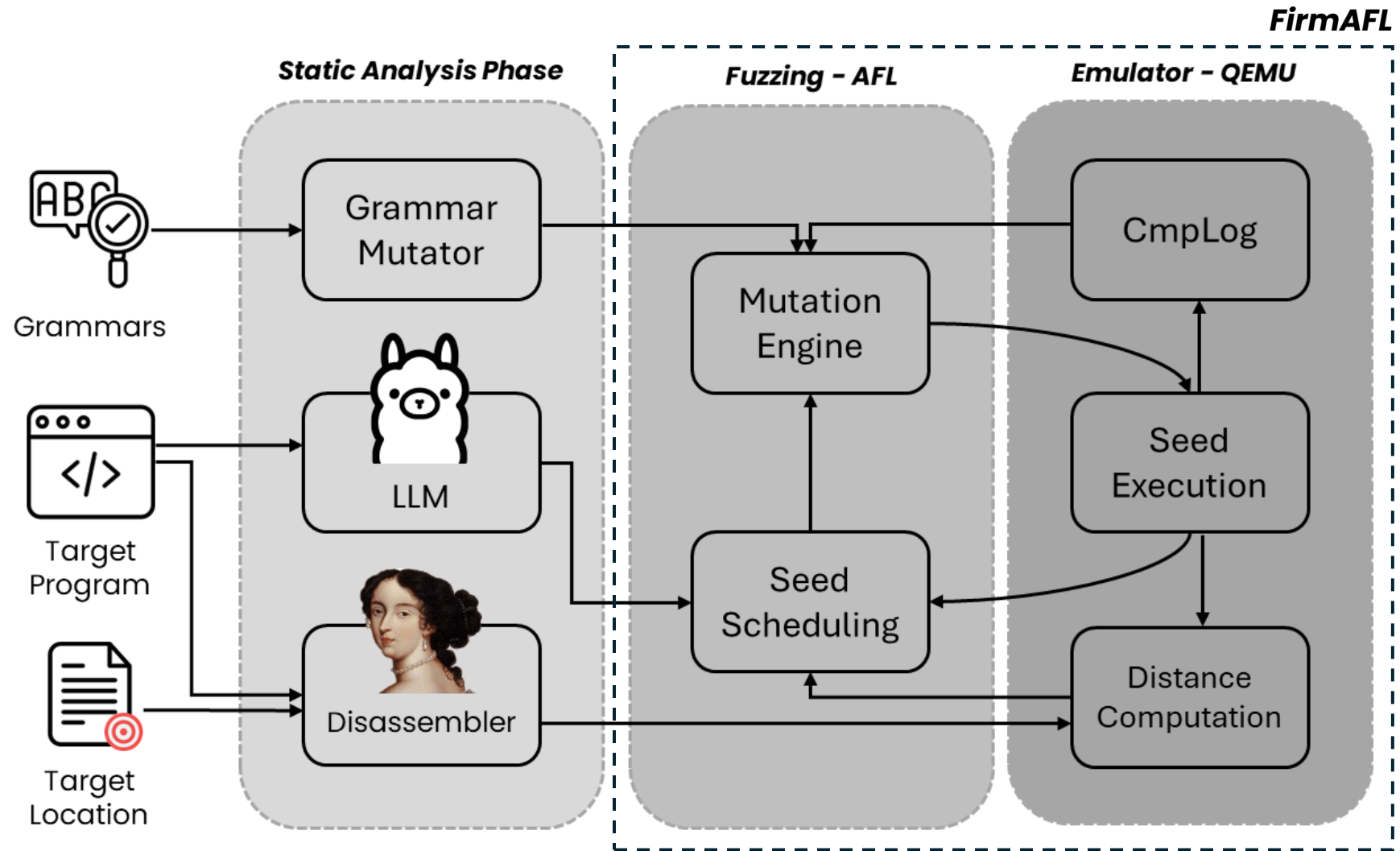
Semantic metric that measures how close an input is to satisfying a branch condition.

Its inclusion in the distance computation mechanism helps reduce wasted effort by considering data-flow aspects when prioritizing inputs.

Conformance is used to weight each proximity score as:

$$d_w = score \times (1 - conformance)$$

Bullseye: Implementation



Evaluation - Experimental Setup

Benchmark:

- 16 CVEs affecting MIPS firmware images.

Evaluation Criteria:

- Median Time-to-Exposure (TTE) of 3 iterations with a 24 hours timeout.

Evaluated Configurations:

	Augmented Input Generation	Directed
Firm-AFL	NO	NO
Bullseye ⁻	YES	NO
Bullseye	YES	YES

Evaluation - Crash Reproduction



Bullseye outperforms baseline across **all 16 cases**, achieving an average speed-up of **6x**.



Evaluation - Case Studies

ID-24926

Bullseye

12 minutes

Why slower? Loop in control-flow

- Target component: `cgibin`.
- Vulnerability: **Buffer Overflow** in uid cookie.

Bullseye⁻

10 minutes

Firm-AFL

80 minutes

2016-1158

Bullseye

5 minutes

- Target component: `httpd`.
- Vulnerability: **Buffer Overflow** in dlink_uid cookie.

Bullseye⁻

26 minutes

Firm-AFL

147 minutes

Conclusions & Open Challenges

Conclusions

- ✓ Improved fuzzing inputs validity.
- ✓ Integrated DGF into the constrained firmware analysis domain.
- ✓ Refined traditional DGF heuristics for more accurate guidance.

Open Challenges

DGF still struggles with complex control flow structures → pair it up with other techniques.

Limited evaluation scope → should be extended to more diverse firmware targets

Analysis bottlenecks → LLMs could be further integrated into the workflow.

State-of-the-Art and Research Directions



Single-service rehosting

Stop trying to emulate the whole device. Extract and run one firmware service in user-space – skip the kernel, skip the peripherals.

REPRESENTATIVE WORK

Greenhouse



Service-Aware Fuzzing

Firmware daemons spawn child processes and speak custom protocols. Drive the fuzzer from service semantics, not raw bytes.

REPRESENTATIVE WORK

HouseFuzz



LLM-Assisted Fuzzing

LLMs infer protocol grammars, generate seed corpora, and synthesize fuzz drivers from documentation or binaries.

REPRESENTATIVE WORK

Fuzz4All, Matter-LLM

STILL OPEN

Even state-of-the-art rehosting covers < 50% of real firmware. Non-Linux RTOS (Real-Time OS) targets, proprietary protocols, and peripheral-heavy devices remain mostly out of reach.

References

- [1] Tay, Hui Jun, et al. "**Greenhouse:{single-service} rehosting of {linux-based} firmware binaries in {user-space} emulation.**" 32nd USENIX Security Symposium (USENIX Security 23). 2023.
- [2] Xiao, Haoyu, et al. "**Housefuzz: Service-aware grey-box fuzzing for vulnerability detection in Linux-based firmware.**" 2025 IEEE Symposium on Security and Privacy (SP). IEEE, 2025.
- [3] Xia, Chunqiu Steven, et al. "**Fuzz4all: Universal fuzzing with large language models.**" Proceedings of the IEEE/ACM 46th International Conference on Software Engineering. 2024.
- [4] Ma, Xiaoyue, Lannan Luo, and Qiang Zeng. "**From one thousand pages of specification to unveiling hidden bugs: Large language model assisted fuzzing of matter {IoT} devices.**" 33rd USENIX Security Symposium (USENIX Security 24). 2024.

Contacts

Email: lorenzo.ralli@uniroma1.it

Linkedin: [lorenzoralli1337](#)

**Thank you for
listening!**